

Software Requirements Specifications

Student Education Fundraising Platform

Project Code: SEFP-2025-001

Internal Advisor: Sir Faisal Shahzad

External Advisor: TBD

Project Manager: Dr. Ilyas

Project Team:

- Team Lead: Umme Habiba BSCS51F22S042
- Team Member 1: Muhammad Kashif BSCS51F22S020
- Team Member 2: Ahmad Jamal BSCS51F22S025

Submission Date: October 20, 2025

Project Manager's Signature

Document Information

Category	Information
Customer	
Project	Student Education Fundraising Platform
Document	Software Requirements Specifications
Document Version	1.0
Identifier	SEFP-2025-001-SRS
Status	Draft
Author(s)	Umme Habiba, Muhammad Kashif, Ahmad Jamal
Approver(s)	Dr. Ilyas (Project Manager)

Issue Date October 6, 2025

Document Location Project Repository/Documentation Folder

Distribution:

1. Project Advisor (Sir Faisal Shahzad)
 2. Project Manager (Dr. Ilyas)
 3. Project Team Members
-

Definition of Terms, Acronyms and Abbreviations

Term	Description
ACH	Automated Clearing House - Electronic network for financial transactions
AI	Artificial Intelligence
API	Application Programming Interface
AWS	Amazon Web Services
CRUD	Create, Read, Update, Delete
ERD	Entity Relationship Diagram
JWT	JSON Web Token
KPI	Key Performance Indicator
KYC	Know Your Customer
LLM	Large Language Model
ML	Machine Learning
NLP	Natural Language Processing
OAuth	Open Authorization
OCR	Optical Character Recognition
PCI DSS	Payment Card Industry Data Security Standard
REST	Representational State Transfer

SEFP	Student Education Fundraising Platform
SQL	Structured Query Language
SRS	Software Requirements Specifications
SSL	Secure Sockets Layer
TLS	Transport Layer Security
UI/UX	User Interface/User Experience
Wire Transfer	Electronic fund transfer between financial institutions

Table of Contents

1. [Introduction](#)
 - 1.1 Purpose of Document
 - 1.2 Project Overview
 - 1.3 Scope
 2. [Overall System Description](#)
 - 2.1 User Characteristics
 - 2.2 Operating Environment
 - 2.3 System Constraints
 3. [External Interface Requirements](#)
 - 3.1 Hardware Interfaces
 - 3.2 Software Interfaces
 - 3.3 Communications Interfaces
 4. [Functional Requirements](#)
 5. [Non-functional Requirements](#)
 - 5.1 Performance Requirements
 - 5.2 Safety Requirements
 - 5.3 Security Requirements
 - 5.4 User Documentation
 6. [Assumptions and Dependencies](#)
 7. [References](#)
 8. [Appendices](#)
-

1. Introduction

1.1 Purpose of Document

This Software Requirements Specifications (SRS) document provides a comprehensive description of the requirements for the Student Education Fundraising Platform. The document is intended for the following audiences:

- **Project Team:** To understand the detailed functional and non-functional requirements for system development
- **Project Manager:** To track project scope, validate deliverables, and ensure alignment with project objectives
- **Project Advisor:** To review and approve the technical specifications and system design
- **Future Developers/Maintainers:** To understand the system architecture and requirements for future enhancements

This document serves as the foundation for system design, development, testing, and deployment phases of the project.

1.2 Project Overview

The Student Education Fundraising Platform is a web-based crowdfunding application designed to connect students in need of educational funding with donors willing to support educational advancement. The platform addresses the critical gap in educational financing by providing a transparent, secure, and trustworthy ecosystem for educational crowdfunding.

Key Innovation: The platform implements an institutional disbursement model where all raised funds are transferred directly to the student's educational institution via automated bank transfers rather than to the student personally. This ensures that funds are used exclusively for legitimate educational expenses and significantly reduces fraud risk.

Primary Goals:

- Reduce financial barriers to education for students worldwide
- Build trust between donors and recipients through institutional involvement
- Provide an AI-powered campaign creation assistant to help students craft compelling fundraising campaigns
- Ensure transparency in fund allocation and usage
- Simplify the donation process for donors with multiple payment options
- Automate fund disbursement to educational institutions

Target Users:

- Students seeking financial assistance for education
- Individual donors wanting to support education
- Institutional donors (corporations, foundations, alumni associations)
- Platform administrators managing campaigns and transfers
- Educational institutions receiving funds

1.3 Scope

The system **WILL**:

1. **User Management:**

- Provide registration and authentication for students and donors
- Implement role-based access control (student, donor, admin)
- Verify student enrollment through document upload and admin review
- Manage user profiles and account security

2. **Campaign Management:**

- Enable students to create, edit, and manage fundraising campaigns
- Provide AI-powered campaign writing assistance using GPT-4 or Claude API
- Implement admin approval workflow for all campaigns
- Support rich media content (images, videos, documents)
- Track campaign progress and milestones

3. **Payment Processing:**

- Process secure donations through Stripe payment gateway
- Support one-time and recurring donations
- Handle multiple currencies
- Generate tax receipts for donors(done by stripe)
- Hold funds in platform escrow until campaign completion

4. **Fund Disbursement:**

- Automate bank transfers (ACH/Wire) to educational institutions
- Track transfer status and confirmations
- Generate transfer reports and audit trails

5. **Admin Dashboard:**

- Provide campaign review and approval interface
- Manage student verification process
- Process fund transfers to universities
- Monitor platform performance and analytics

6. **Communication Features:**

- Send email notifications for campaign updates
- Enable social media sharing
- Allow campaign commenting and interaction
- Facilitate thank-you messages from students to donors

7. **Analytics and Reporting:**

- Display campaign performance metrics

- Show donor contribution history
- Generate platform-wide impact reports
- Provide financial reconciliation reports

The system WILL NOT:

1. Allow students to withdraw funds directly (all funds go to institutions)
 2. Support mobile applications in version 1.0 (web responsive design only)
 3. Implement direct messaging between students and donors
 4. Integrate with external scholarship databases
 5. Support cryptocurrency payments
 6. Provide multi-language support (English only initially)
 7. Offer peer-to-peer lending or loan facilities
 8. Include video call functionality for verification
 9. Integrate with institutional ERP/LMS systems (manual process)
 10. Support real-time bank transfers (batch processing only)
-

2. Overall System Description

2.1 User Characteristics

The system will serve four distinct user classes:

2.1.1 Students (Primary Users)

- **Technical Expertise:** Basic to intermediate computer literacy
- **Education Level:** High school graduates to graduate students
- **Age Range:** 16-30 years
- **Needs:**
 - Easy campaign creation process
 - Clear guidance on writing compelling stories
 - Transparency on fund status and transfers
 - Ability to share campaigns on social media
- **Frequency of Use:** Moderate (active during campaign creation and monitoring)
- **Critical Requirements:**
 - Simple, intuitive interface
 - Mobile-responsive design (web-based)
 - Clear instructions and help resources
 - AI assistance for campaign writing

2.1.2 Donors (Primary Users)

- **Technical Expertise:** Basic to intermediate computer literacy

- **Education Level:** Varied
- **Age Range:** 25-65 years
- **Needs:**
 - Easy browsing and search for campaigns
 - Transparency on fund usage
 - Secure payment processing
 - Tax receipts for donations
 - Assurance that funds reach intended purpose
- **Frequency of Use:** Low to moderate (occasional donations)
- **Critical Requirements:**
 - Trust indicators (verification badges, institutional backing)
 - Simple donation process
 - Multiple payment options
 - Clear impact reporting

2.1.3 Platform Administrators (Critical Users)

- **Technical Expertise:** Advanced computer literacy
- **Education Level:** College degree or higher
- **Age Range:** 25-50 years
- **Needs:**
 - Efficient campaign review and approval tools
 - Student verification interface
 - Fund transfer management
 - Fraud detection and monitoring
 - Platform analytics and reporting
- **Frequency of Use:** Daily (full-time platform management)
- **Critical Requirements:**
 - Comprehensive admin dashboard
 - Quick review and approval workflows
 - Bank transfer processing tools
 - Audit trail and compliance reporting

2.1.4 Educational Institutions (Secondary Users)

- **Technical Expertise:** Not applicable (receive bank transfers only)
- **Education Level:** Not applicable
- **Interaction:** Minimal (receive funds and confirmation emails)
- **Needs:**
 - Clear fund transfer notifications
 - Student identification information
 - Purpose of funds documentation
- **Frequency of Use:** Passive (receive notifications)
- **Critical Requirements:**
 - Accurate transfer information

- Proper documentation for internal accounting

2.2 Operating Environment

2.2.1 Client-Side Environment

- **Platform:** Web browsers (Chrome, Firefox, Safari, Edge)
- **Browser Versions:** Latest 2 major versions
- **Device Types:** Desktop computers, laptops, tablets, smartphones
- **Screen Resolutions:** Responsive design supporting 320px to 4K displays
- **Internet Connection:** Broadband or mobile data (minimum 1 Mbps recommended)
- **JavaScript:** Must be enabled
- **Cookies:** Must be enabled for session management

2.2.2 Server-Side Environment

- **Cloud Platform:** AWS (Amazon Web Services)
- **Web Server:** Node.js with Express.js framework
- **Application Architecture:** Microservices architecture
- **Database Server:** PostgreSQL 14 or higher
- **Cache Server:** Redis 6.x or higher
- **File Storage:** AWS S3
- **CDN:** AWS CloudFront for static asset delivery
- **Operating System:** Ubuntu Server 20.04 LTS or higher
- **Containerization:** Docker (optional for deployment)

2.2.3 Third-Party Services

- **Payment Gateway:** Stripe API
- **Bank Transfers:** Stripe Payouts or Plaid
- **AI Services:** OpenAI GPT-4 or Anthropic Claude
- **Email Service:** SendGrid, Resend, or AWS SES
- **Error Tracking:** Sentry
- **Analytics:** Google Analytics (optional)

2.3 System Constraints

2.3.1 Software Constraints

- Must use React.js with Next.js for frontend development
- Must use Node.js with Express.js for backend development
- Must use PostgreSQL for relational data storage
- Must integrate with Stripe API for payment processing

- Must support only web browsers (no native mobile apps in v1.0)
- Must operate in English language only initially

2.3.2 Hardware Constraints

- Minimum server specifications:
 - 4 CPU cores
 - 8 GB RAM
 - 100 GB SSD storage
 - 1 Gbps network connection
- Scalable infrastructure to handle growing user base
- CDN for global content delivery

2.3.3 Legal Constraints

- Must comply with data protection regulations (GDPR, CCPA)
- Must comply with financial regulations for fund transfers
- Must adhere to anti-money laundering (AML) regulations
- Must provide terms of service and privacy policy
- Must obtain user consent for data processing
- Must comply with educational verification regulations

2.3.4 Cultural and Language Constraints

- Initial version supports English language only
- Currency support limited to countries supported by Stripe
- Cultural considerations for campaign content moderation

2.3.5 Environmental Constraints

- Must operate in standard office/home environments
- No special environmental requirements for users
- Standard internet connectivity required
- No accessibility to high-security networks needed

2.3.6 User Constraints

- Students must be enrolled in recognized educational institutions
- Students must be able to provide enrollment verification documents
- Donors must have valid payment methods
- Users must have email addresses for registration
- Users must be comfortable with web-based applications
- Admin users must have banking knowledge for transfer processing

2.3.7 Regulatory and Compliance Constraints

- Must maintain audit trails for all financial transactions
- Must generate tax documentation for donations
- Must verify student identities before campaign approval
- Must ensure funds are used for educational purposes only
- Must comply with banking regulations for fund transfers
- Must implement KYC (Know Your Customer) procedures

2.3.8 Technical Constraints

- Bank transfers processed in batches (not real-time)
- University banking information must be collected manually
- Student verification is manual by admin team
- AI API rate limits may affect concurrent campaign creations
- Payment processing limited to Stripe-supported countries
- Maximum file upload size: 10 MB per file
- Campaign images limited to 5 per campaign
- Video uploads must be hosted externally (YouTube/Vimeo links only)

2.3.9 Budget Constraints

- Cloud infrastructure costs must be optimized
- AI API usage must be monitored to control costs
- Payment processing fees are 2.9% + \$0.30 per transaction
- Bank transfer fees: ACH (\$0.20-\$1.50), Wire (\$15-\$30)
- Limited budget for third-party integrations

2.3.10 Time Constraints

- Project must be completed within 19 weeks
- MVP must be ready for testing by week 16
- Beta testing period: 2 weeks
- Production deployment by week 18

3. External Interface Requirements

3.1 Hardware Interfaces

3.1.1 Client Hardware

The system will interface with various client devices through standard web browsers. No special hardware drivers or interfaces are required.

Supported Devices:

- Desktop computers (Windows, macOS, Linux)
- Laptops (Windows, macOS, Linux)
- Tablets (iOS, Android, Windows)
- Smartphones (iOS, Android)

Hardware Requirements:

- Display: Minimum 320px width, responsive up to 4K
- Input: Mouse, keyboard, or touch screen
- Network: Wi-Fi or cellular data connection
- Camera: Optional (for document scanning via mobile browser)

3.1.2 Server Hardware

The system will be deployed on cloud infrastructure (AWS) with the following specifications:

Production Environment:

- EC2 instances: t3.medium or higher (2 vCPU, 4 GB RAM minimum)
- RDS Database: db.t3.medium or higher
- S3 Storage: Unlimited scalable storage
- Load Balancer: Application Load Balancer (ALB)
- Auto Scaling: Enabled for handling traffic spikes

Backup Systems:

- S3 bucket replication: Cross-region
- Disaster recovery: Multi-availability zone deployment

3.1.3 Payment Processing Hardware

All payment processing is handled by Stripe's infrastructure. No special hardware interfaces are required on the client or server side.

3.2 Software Interfaces

3.2.1 Operating System Interfaces

Client Side:

- No direct OS interface required
- Standard web browser compatibility

Server Side:

- Operating System: Ubuntu Server 20.04 LTS
- Package Manager: apt (for system packages), npm (for Node.js packages)
- Process Manager: PM2 for Node.js application management
- System Services: systemd for service management

3.2.2 Database Interface

PostgreSQL Database:

- Version: 14.x or higher
- Connection: TCP/IP over port 5432
- Protocol: PostgreSQL wire protocol
- Driver: node-postgres (pg) npm package
- Connection Pooling: pgBouncer or built-in pooling
- Data Format: SQL queries, JSON for structured data
- Backup: pg_dump for database backups

3.2.3 Payment Gateway Interface

Stripe API:

- API Version: Latest stable version (2024-10-15 or newer)
- Authentication: API keys (secret and publishable)
- Protocol: HTTPS REST API
- Data Format: JSON
- SDK: stripe npm package (Node.js)

Key Integrations:

- Payment Intents API: For processing donations
- Stripe Webhooks: For payment status updates
- Stripe Payouts API: For bank transfers to universities
- Stripe Identity: For KYC verification (optional)

Data Exchanged:

- Payment information (amount, currency, metadata)
- Customer information (email, name)
- Payment method details (tokenized, no raw card data stored)
- Webhook events (payment succeeded, failed, refunded)
- Payout information (transfer amounts, bank details)

3.2.4 AI Service Interfaces

OpenAI GPT-4 API or Anthropic Claude API:

- API Version: Latest stable
- Authentication: API key
- Protocol: HTTPS REST API
- Data Format: JSON
- Purpose: Campaign content generation and assistance

Data Exchanged:

- Prompt text: User's campaign draft or outline
- Response text: AI-generated suggestions and improvements
- Metadata: Model selection, temperature, max tokens
- Error messages: Rate limit, quota exceeded, etc.

3.2.5 Email Service Interface

SendGrid / Resend / AWS SES:

- API Version: Latest stable
- Authentication: API key or AWS credentials
- Protocol: HTTPS REST API or SMTP
- Data Format: JSON (REST) or MIME (SMTP)
- Purpose: Transactional emails (notifications, receipts, alerts)

Email Types:

- User registration confirmation
- Campaign approval/rejection notifications
- Donation receipts
- Transfer completion notifications
- Password reset emails
- Campaign update announcements

3.2.6 Cloud Storage Interface

AWS S3:

- API Version: AWS SDK v3
- Authentication: AWS IAM credentials
- Protocol: HTTPS REST API
- Data Format: Multipart uploads for large files
- Purpose: Storing campaign images, documents, and user uploads

File Operations:

- Upload: Multipart upload for files > 5 MB
- Download: Presigned URLs for secure access
- Delete: Cascade delete on campaign removal
- Access Control: Bucket policies and IAM roles

3.2.7 Monitoring and Error Tracking

Sentry:

- API Version: Latest SDK
- Authentication: DSN (Data Source Name)
- Protocol: HTTPS
- Data Format: JSON
- Purpose: Error tracking, performance monitoring

Monitored Events:

- Application errors and exceptions
- API failures
- Performance bottlenecks
- User actions leading to errors

3.3 Communications Interfaces

3.3.1 HTTP/HTTPS Protocol

- **Protocol Version:** HTTP/1.1, HTTP/2
- **Encryption:** TLS 1.2 or higher
- **Certificates:** SSL certificates from Let's Encrypt or AWS Certificate Manager
- **Port:** 443 (HTTPS)
- **Data Format:** JSON for API responses, HTML for page rendering

3.3.2 WebSocket Protocol (Optional for Real-time Features)

- **Protocol:** WebSocket (ws:// or wss://)
- **Purpose:** Real-time notifications, live campaign updates
- **Library:** Socket.io (if implemented)
- **Fallback:** Long polling for unsupported browsers

3.3.3 RESTful API Communication

3.3.4 Email Communication

- **Protocol:** SMTP or HTTPS (for API-based services)
- **Port:** 587 (TLS), 465 (SSL), or HTTPS

- **Authentication:** API key or SMTP credentials
- **Format:** HTML and plain text fallback
- **Rate Limiting:** Respect ESP rate limits

3.3.5 Third-Party API Communication

Security Requirements:

- All API calls over HTTPS
- API keys stored in environment variables
- Rate limiting implementation
- Error handling and retry logic
- Request logging for audit trails

Message Formatting:

- Standard JSON format for all API requests/responses
- ISO 8601 format for timestamps
- UTF-8 encoding for all text data
- Base64 encoding for binary data (images, documents)

3.3.6 Network Requirements

- **Bandwidth:** Minimum 100 Mbps for server
- **Latency:** Target < 500ms for API responses
- **Uptime:** 99.9% availability target
- **Concurrent Connections:** Support up to 100 simultaneous users

3.3.7 Data Synchronization

- **Real-time Updates:** Campaign progress, donation notifications
- **Batch Processing:** Bank transfers, email notifications
- **Conflict Resolution:** Last-write-wins for concurrent edits
- **Data Consistency:** ACID compliance for financial transactions

4. Functional Requirements

4.1 User Management

FR-UM-001: User Registration

Priority: High

Description: The system shall allow users to register as students or donors.

Inputs:

- Email address
- Password
- User type (student/donor)
- Full name
- Optional: Phone number

Processing:

- Validate email format and uniqueness
- Encrypt password using bcrypt
- Generate verification token
- Send verification email

Outputs:

- User account created
- Verification email sent
- Success/failure message

Acceptance Criteria:

- Password must be minimum 8 characters with uppercase, lowercase, and number
- Email must be unique in the system
- Verification email must be sent within 1 minute

FR-UM-002: Email Verification

Priority: High

Description: Users must verify their email address before accessing the platform.

Inputs:

- Verification token from email link

Processing:

- Validate token expiry (24 hours)
- Mark email as verified
- Enable account access

Outputs:

- Account activated
- Redirect to login page

FR-UM-003: User Login

Priority: High

Description: Registered users shall be able to log in to access their accounts.

Inputs:

- Email address
- Password

Processing:

- Validate credentials
- Generate JWT token
- Create session

Outputs:

- JWT token
- User profile data
- Redirect to dashboard

Acceptance Criteria:

- Invalid credentials show appropriate error message
- Account must be verified to login
- Session expires after 7 days of inactivity

FR-UM-004: OAuth 2.0 Social Login

Priority: Medium

Description: Users shall be able to register/login using social media accounts.

Supported Providers:

- Google
- Facebook (optional)

Processing:

- OAuth callback handling
- Account linking or creation
- Profile data synchronization

FR-UM-005: Password Reset

Priority: High

Description: Users shall be able to reset forgotten passwords.

Inputs:

- Email address

Processing:

- Generate password reset token
- Send reset email with link
- Validate token on reset page
- Update password

Outputs:

- Password reset email sent
- Password updated successfully

Acceptance Criteria:

- Reset link expires after 1 hour
- Old password is invalidated immediately

FR-UM-006: User Profile Management

Priority: Medium

Description: Users shall be able to view and edit their profile information.

Editable Fields:

- Full name
- Phone number
- Profile picture
- Bio/About section
- Notification preferences

Non-editable Fields:

- Email (requires verification to change)
- User type
- Registration date

FR-UM-007: Student Verification

Priority: High

Description: Students must upload verification documents for enrollment confirmation.

Inputs:

- Student ID document (image/PDF)
- Enrollment letter (image/PDF)
- University name (dropdown selection)
- Student ID number
- Program/Major
- Expected graduation date

Processing:

- Store documents in S3
- Create verification request
- Notify admin team
- Admin reviews documents
- Approve or reject with reason

Outputs:

- Verification status (pending/approved/rejected)
- Notification to student

Acceptance Criteria:

- Maximum file size: 10 MB per document
- Supported formats: JPG, PNG, PDF
- Admin review within 48 hours

FR-UM-008: Admin User Management

Priority: High

Description: Admin shall be able to manage all user accounts.

Admin Capabilities:

- View all users (students, donors, admins)
- Search and filter users
- View user details and activity
- Suspend/unsuspend accounts
- Delete accounts (with confirmation)
- Assign admin roles

4.2 Campaign Management

FR-CM-001: Campaign Creation

Priority: High

Description: Verified students shall be able to create fundraising campaigns.

Inputs:

- Campaign title (max 100 characters)
- Campaign description (rich text, max 5000 characters)
- Funding goal (amount in USD)
- Campaign deadline (date)
- Category (dropdown: tuition, books, accommodation, equipment, fees)
- University (auto-filled from student profile)
- Campaign images (up to 5 images)
- Optional: Video link (YouTube/Vimeo)

Processing:

- Validate all inputs
- Save as draft status
- Store images in S3
- Generate campaign ID
- Create campaign record

Outputs:

- Campaign saved as draft
- Success message
- Option to submit for review

Acceptance Criteria:

- Funding goal must be between \$50 and \$50,000
- Deadline must be 7-90 days in future
- At least one image required

FR-CM-002: AI Campaign Assistant

Priority: High

Description: System shall provide AI-powered assistance for campaign content creation.

Inputs:

- Student's basic information
- Campaign purpose (brief outline)
- Target amount and reason

Processing:

- Call GPT-4 or Claude API
- Generate campaign title suggestions
- Generate campaign description draft
- Improve grammar and readability
- Suggest emotional hooks

Outputs:

- AI-generated content suggestions
- Multiple title options
- Formatted description

Acceptance Criteria:

- AI response within 10 seconds
- User can edit AI suggestions
- User can regenerate content
- Usage tracked for cost control

FR-CM-003: Campaign Submission for Approval

Priority: High

Description: Students shall submit completed campaigns for admin review.

Inputs:

- Campaign ID
- Submission confirmation

Processing:

- Change status from draft to pending_review
- Notify admin team
- Lock campaign from editing

Outputs:

- Status updated
- Admin notification sent
- Student receives confirmation

FR-CM-004: Campaign Review by Admin

Priority: High

Description: Admin shall review and approve/reject submitted campaigns.

Admin Review Interface:

- View campaign details
- View student verification status
- View uploaded documents
- AI fraud score (if available)
- Campaign history

Admin Actions:

- Approve campaign (goes live)
- Reject campaign (with reason)
- Request modifications

Outputs:

- Campaign status updated (approved/rejected)
- Student notified with reason (if rejected)
- Campaign published (if approved)

Acceptance Criteria:

- Review completed within 48 hours
- Rejection reason must be provided
- Student can resubmit after modifications

FR-CM-005: Campaign Display and Browsing

Priority: High

Description: Donors shall be able to browse and view active campaigns.

Browse Features:

- List all active campaigns
- Filter by category
- Filter by university
- Filter by amount needed
- Filter by urgency (deadline approaching)
- Sort by date created, amount raised, urgency

Campaign Detail View:

- Full campaign story
- Images and videos
- Funding progress bar
- Amount raised / Goal amount
- Days remaining
- Donor count

- Recent donations (anonymous option respected)
- University verification badge
- Comment section

FR-CM-006: Campaign Search

Priority: Medium

Description: Users shall be able to search for campaigns by keywords.

Search Capabilities:

- Search by campaign title
- Search by student name
- Search by university
- Search by category
- Auto-suggest as user types

Search Results:

- Ranked by relevance
- Display campaign card with preview
- Filter and sort options

FR-CM-007: Campaign Updates

Priority: Medium

Description: Students shall be able to post updates on their campaigns.

Update Features:

- Post text updates
- Upload new images
- Notify donors via email
- Display updates chronologically

Inputs:

- Update text (max 1000 characters)
- Optional image

Outputs:

- Update posted on campaign page
- Email notification to donors (opt-in)

FR-CM-008: Campaign Sharing

Priority: Medium

Description: Users shall be able to share campaigns on social media.

Supported Platforms:

- Facebook
- Twitter
- LinkedIn
- WhatsApp
- Email
- Copy link

Sharing Features:

- Pre-filled text with campaign details
- Open Graph meta tags for rich previews
- Track shares for analytics

FR-CM-010: Campaign Analytics

Priority: Medium

Description: Students shall be able to view campaign performance analytics.

Analytics Dashboard:

- Total amount raised
- Number of donors
- Average donation amount
- Donation timeline (chart)
- Traffic sources
- Social shares count
- Page views

4.3 Payment Processing

FR-PP-001: Donation Processing

Priority: High

Description: System shall process secure donations via Stripe.

Inputs:

- Campaign ID
- Donation amount (min \$50, max \$10,000)
- Payment method (credit/debit card)
- Optional: Donor message

- Optional: Donate anonymously (checkbox)
- Optional: Recurring donation (monthly)

Processing:

- Validate amount
- Create Stripe Payment Intent
- Process payment via Stripe
- Calculate platform fee (10%)
- Calculate Stripe fee (2.9% + \$0.30)
- Update campaign raised amount
- Create donation record

Outputs:

- Payment confirmation
- Donation receipt email
- Thank you page
- Update campaign progress

Acceptance Criteria:

- Payment processing within 5 seconds
- Failed payments show user-friendly error
- Funds held in platform Stripe account
- Receipt sent immediately

FR-PP-002: Recurring Donations

Priority: Medium

Description: Donors shall be able to set up monthly recurring donations.

Inputs:

- Recurring amount
- Start date
- Optional: End date or number of payments

Processing:

- Create Stripe subscription
- Store subscription ID
- Process monthly payments automatically
- Send monthly receipts

Outputs:

- Subscription confirmation
- Monthly payment confirmations

Cancellation:

- Donor can cancel anytime
- Email confirmation of cancellation

FR-PP-003: Donation Receipt Generation

Priority: High

Description: System shall generate tax-deductible donation receipts.

Receipt Contents:

- Donor name
- Donation amount
- Date and time
- Campaign name
- Student name (unless anonymous)
- Platform name and tax ID (if applicable)
- Transaction ID

Delivery:

- Email immediately after donation
- Available for download in donor dashboard

FR-PP-004: Refund Processing

Priority: Medium

Description: System shall process refunds for donations in specific cases.

Refund Scenarios:

- Campaign canceled by admin (fraud)
- Campaign deadline expired without reaching minimum
- Donor dispute (within 30 days)

Processing:

- Admin initiates refund
- Process through Stripe
- Update campaign amount
- Notify donor

Acceptance Criteria:

- Refunds processed within 5-10 business days
- Platform fee refunded
- Stripe fee non-refundable

FR-PP-005: Payment Security

Priority: High

Description: System shall maintain PCI DSS compliance for payment processing.

Security Measures:

- No card data stored on platform servers
- Stripe.js for client-side tokenization
- HTTPS for all payment pages
- Secure payment form rendering

4.4 Fund Disbursement

FR-FD-001: Bank Transfer Initiation

Priority: High

Description: Admin shall initiate bank transfers to universities when campaigns complete.

Transfer Trigger:

- Campaign reaches goal OR deadline expires
- Campaign has raised at least minimum amount (\$100)
- Admin reviews campaign completion

Inputs:

- Campaign ID
- University banking information (from database)
- Transfer amount (raised amount - fees)

Processing:

- Calculate net amount (gross - platform fee - Stripe fees - transfer fee)
- Verify university banking details
- Create transfer record
- Initiate ACH or Wire transfer via Stripe Payouts or Plaid
- Update campaign status to "transferred"

Outputs:

- Transfer initiated

- Transfer confirmation
- Email to university
- Email to student
- Admin notification

Acceptance Criteria:

- Transfer initiated within 48 hours of campaign completion
- ACH transfers: 2-3 business days
- Wire transfers: 1 business day
- Transfer confirmation tracked

FR-FD-002: Transfer Tracking

Priority: High

Description: System shall track all bank transfer statuses.

Transfer Statuses:

- Pending: Transfer scheduled but not sent
- Processing: Transfer in progress
- Completed: Funds received by university
- Failed: Transfer failed (retry needed)

Tracking Information:

- Transfer date and time
- Amount transferred
- University details
- Confirmation number
- Status updates

Notifications:

- Admin notified on status change
- Student notified when completed
- University notified on completion

FR-FD-003: Transfer Reporting

Priority: Medium

Description: System shall generate transfer reports for auditing.

Report Contents:

- Campaign details
- Student information

- University details
- Gross amount raised
- Platform fee deducted
- Stripe fees deducted
- Transfer fee deducted
- Net amount transferred
- Transfer date
- Confirmation number

Report Formats:

- PDF for download
- CSV for export
- Email delivery

Access:

- Admin can generate reports
- Filter by date range, university, status

FR-FD-004: Failed Transfer Handling

Priority: High

Description: System shall handle failed bank transfers appropriately.

Failure Scenarios:

- Invalid bank account number
- Bank account closed
- Insufficient platform funds (edge case)
- Network issues

Handling Process:

- Retry automatically (up to 3 times)
- Alert admin after 3 failures
- Admin manually resolves issue
- Update university banking info if needed
- Retry manual transfer

Notifications:

- Admin alerted immediately
- Student informed of delay
- University contacted for correct info

4.5 Admin Dashboard

FR-AD-001: Admin Login and Authentication

Priority: High

Description: Admin users shall have secure access to admin dashboard.

Authentication:

- Admin-specific login
- Multi-factor authentication (optional)
- Role-based permissions
- Session timeout after 30 minutes inactivity

FR-AD-002: Campaign Management Interface

Priority: High

Description: Admin shall have comprehensive campaign management tools.

Admin Views:

- All campaigns list (all statuses)
- Pending review queue
- Active campaigns
- Completed campaigns
- Rejected campaigns

Admin Actions:

- Review and approve campaigns
- Reject campaigns with reason
- Edit campaign details (if needed)
- Cancel fraudulent campaigns
- View campaign analytics

FR-AD-003: Student Verification Interface

Priority: High

Description: Admin shall review and verify student documents.

Verification Queue:

- List of pending verifications
- Student details
- Uploaded documents viewer
- University information

Admin Actions:

- Approve verification
- Reject verification with reason
- Request additional documents
- Mark as suspicious

Document Viewer:

- Display images inline
- PDF viewer
- Zoom and pan capabilities
- Download option

FR-AD-004: University Database Management

Priority: High

Description: Admin shall manage university information and banking details.

University Records:

- University name
- Location (city, state, country)
- Contact information
- Banking details (encrypted)
 - Bank name
 - Account number (encrypted)
 - Routing number
 - Account holder name

Admin Actions:

- Add new university
- Edit university information
- Update banking details
- Verify university authenticity

Security:

- Banking information encrypted at rest
- Only admins with specific role can view full account numbers
- Audit log for all banking info access

FR-AD-005: Transfer Processing Interface

Priority: High

Description: Admin shall process fund transfers to universities.

Transfer Queue:

- Campaigns ready for transfer
- Campaign details
- Calculated transfer amount
- University banking info

Transfer Actions:

- Review transfer details
- Initiate transfer
- Cancel transfer (if needed)
- Retry failed transfer

Transfer Methods:

- ACH (automated via Stripe Payouts)
- Wire transfer (manual or automated)
- Mark as manually processed

FR-AD-006: Platform Analytics Dashboard

Priority: Medium

Description: Admin shall view platform-wide analytics and metrics.

Metrics Displayed:

- Total campaigns created
- Total campaigns approved/rejectedd
- Total funds raised
- Total funds transferred
- Number of active donors
- Number of verified students
- Average campaign success rate
- Platform fee revenue
- User growth over time

Visualizations:

- Charts and graphs
- Date range filters
- Export to PDF/CSV

FR-AD-007: User Support Interface

Priority: Medium

Description: Admin shall handle user support requests and disputes.

Support Features:

- View all user tickets/issues
- Respond to user inquiries
- Escalate issues
- View user activity logs
- Suspend/unsuspend accounts

4.6 Communication and Notifications

FR-CN-001: Email Notifications

Priority: High

Description: System shall send email notifications for important events.

Notification Types:

For Students:

- Account verification email
- Campaign approved/rejected
- Donation received
- Campaign milestone reached (25%, 50%, 75%, 100%)
- Campaign deadline approaching (7 days, 3 days, 1 day)
- Transfer initiated to university
- Transfer completed

For Donors:

- Donation receipt
- Campaign completion (campaign they donated to)
- Thank you message from student
- Recurring donation processed
- Recurring donation failed

For Admins:

- New campaign submitted for review
- New student verification request
- Transfer ready to process
- Failed transfer alert

- Suspicious activity detected

Email Features:

- HTML and plain text versions
- Responsive design for mobile
- Unsubscribe option (for non-critical emails)
- Delivery tracking

FR-CN-002: In-App Notifications

Priority: Low

Description: Users shall receive in-app notifications.

Notification Center:

- Bell icon with unread count
- List of recent notifications
- Mark as read/unread
- Delete notifications

Notification Types:

- Campaign status changes
- Donations received
- Comments on campaigns
- System announcements

FR-CN-003: Social Media Sharing

Priority: Medium

Description: Users shall be able to share campaigns on social platforms.

Sharing Features:

- Facebook share button
- Twitter share button
- LinkedIn share button
- WhatsApp share button
- Copy link button
- Email share option

Share Content:

- Pre-filled text with campaign details
- Campaign image
- Link to campaign

4.7 Search and Discovery

FR-SD-001: Campaign Search

Priority: High

Description: Users shall be able to search for campaigns.

Search Fields:

- Campaign title
- Student name
- University name
- Category
- Keywords in description

Search Features:

- Auto-complete suggestions
- Search as you type
- Recent searches
- Popular searches

FR-SD-002: Campaign Filtering

Priority: High

Description: Users shall be able to filter campaigns by various criteria.

Filter Options:

- Category (tuition, books, accommodation, etc.)
- University
- Amount needed (ranges)
- Deadline (urgency)
- Location (state/country)
- Verification status

Filter UI:

- Sidebar with checkboxes
- Apply multiple filters
- Clear all filters
- Show result count

FR-SD-003: Campaign Sorting

Priority: Medium

Description: Users shall be able to sort campaign listings.

Sort Options:

- Newest first
- Ending soon (deadline approaching)
- Most funded
- Least funded
- Alphabetical (A-Z)

FR-SD-004: Featured Campaigns

Priority: Low

Description: Admin shall be able to feature campaigns on homepage.

Featured Criteria:

- High-quality story
- Close to goal
- Urgent deadline
- Verified student

Display:

- Homepage banner/carousel
- "Featured" badge on campaign card

4.8 Analytics and Reporting

FR-AR-001: Student Dashboard

Priority: High

Description: Students shall view their campaign performance.

Dashboard Metrics:

- Total raised
- Percentage of goal
- Number of donors
- Days remaining
- Average donation amount
- Page views
- Share count

Visualizations:

- Donation timeline chart
- Traffic sources pie chart
- Progress bar

FR-AR-002: Donor Dashboard

Priority: Medium

Description: Donors shall view their donation history and impact.

Dashboard Metrics:

- Total donated
- Number of campaigns supported
- Number of completed campaigns
- Recent donations list
- Recurring donations list

Features:

- View campaign outcomes
- Download receipts
- Manage recurring donations

FR-AR-003: Admin Analytics

Priority: Medium

Description: Admin shall view comprehensive platform analytics.

Analytics Sections:

- User statistics (growth, active users)
- Campaign statistics (success rate, average goal)
- Financial statistics (funds raised, fees collected)
- Performance metrics (page load times, API response times)

Export:

- PDF reports
- CSV data export
- Scheduled email reports (weekly/monthly)

5. Non-functional Requirements

5.1 Performance Requirements

NFR-PR-001: Response Time

Priority: High

Description: System shall respond to user actions within acceptable time limits.

Requirements:

- Page load time: ≤ 3 seconds (on 3G connection)
- API response time: ≤ 500 milliseconds (average)
- Search results: ≤ 2 seconds
- AI content generation: ≤ 10 seconds
- Payment processing: ≤ 5 seconds
- Database queries: ≤ 100 milliseconds (average)

Measurement:

- Monitor with DataDog or similar
- Track 95th percentile response times
- Set up alerts for performance degradation

NFR-PR-002: Throughput

Priority: High

Description: System shall handle concurrent users and transactions.

Requirements:

- Support 1,000 concurrent users (initial launch)
- Support 10,000 concurrent users (after scaling)
- Process 100 donations per minute
- Handle 1,000 API requests per minute
- Process 10 bank transfers per batch

Scalability:

- Auto-scaling enabled on AWS
- Horizontal scaling for web servers
- Database read replicas for query performance
- CDN for static assets

NFR-PR-003: Resource Utilization

Priority: Medium

Description: System shall efficiently use server resources.

Requirements:

- CPU utilization: $\leq 70\%$ under normal load
- Memory utilization: $\leq 80\%$ under normal load
- Database connections: $\leq 80\%$ of pool size
- Disk I/O: Optimized with caching

Optimization:

- Implement Redis caching for frequent queries
- Use database indexes for common searches
- Optimize images and assets
- Lazy loading for images

NFR-PR-004: Data Volume

Priority: Medium

Description: System shall handle expected data volumes.

Expected Data:

- 1,000 users in first year
- 1,000 campaigns in first year
- 10,000 donations in first year
- 1 TB of storage (images, documents)

Database Design:

- Partitioning for large tables
- Archiving old data
- Regular database optimization

NFR-PR-005: Availability

Priority: High

Description: System shall maintain high availability.

Requirements:

- Uptime: 99.9% (maximum 8.76 hours downtime per year)
- Maintenance windows: Scheduled during low-traffic hours
- Disaster recovery: 4-hour RTO (Recovery Time Objective)
- Data backup: Daily automated backups with 30-day retention

High Availability Setup:

- Multi-availability zone deployment
- Load balancing across multiple servers
- Automated failover

- Health checks and monitoring

5.2 Safety Requirements

NFR-SR-001: Data Backup and Recovery

Priority: High

Description: System shall implement comprehensive backup and recovery procedures.

Backup Requirements:

- Database: Automated daily backups
- File storage: S3 versioning and replication
- Backup retention: 30 days
- Backup testing: Monthly restore tests

Recovery Procedures:

- Point-in-time recovery for database
- Disaster recovery plan documented
- Regular disaster recovery drills
- Maximum data loss: 1 hour (RPO - Recovery Point Objective)

NFR-SR-002: Error Handling

Priority: High

Description: System shall handle errors gracefully without data loss.

Error Handling:

- Try-catch blocks for all critical operations
- Transaction rollback for database errors
- Graceful degradation for third-party API failures
- User-friendly error messages (no technical jargon)
- Detailed error logging for debugging

Critical Operations:

- Payment processing: Retry logic and idempotency
- Bank transfers: Queue-based processing with retries
- File uploads: Chunked uploads with resume capability

NFR-SR-003: Data Integrity

Priority: High

Description: System shall maintain data consistency and integrity.

Integrity Measures:

- ACID compliance for financial transactions
- Foreign key constraints in database
- Input validation on client and server
- Duplicate detection for donations
- Audit trails for critical data changes

Validation:

- Email format validation
- Payment amount range validation
- File type and size validation
- Date range validation

NFR-SR-004: Failover Mechanisms

Priority: High

Description: System shall implement failover for critical services.

Failover Strategy:

- Primary database with read replica
- Automatic failover to secondary server
- Queue-based processing for asynchronous tasks
- Retry logic for failed operations

5.3 Security Requirements

NFR-SEC-001: Authentication and Authorization

Priority: High

Description: System shall implement secure authentication and authorization.

Authentication:

- Password hashing: bcrypt with salt
- Password requirements: Minimum 8 characters, uppercase, lowercase, number
- JWT tokens for session management
- Token expiration: 7 days
- Refresh token mechanism
- OAuth 2.0 for social login

Authorization:

- Role-based access control (RBAC)

- User roles: Student, Donor, Admin
- Permission checks on all protected routes
- Admin-only endpoints secured

NFR-SEC-002: Data Encryption

Priority: High

Description: System shall encrypt sensitive data.

Encryption at Rest:

- Database: AWS RDS encryption enabled
- File storage: S3 server-side encryption
- Banking information: AES-256 encryption
- Encryption keys: AWS KMS (Key Management Service)

Encryption in Transit:

- HTTPS/TLS 1.2 or higher for all communications
- SSL certificates from trusted CA
- HSTS (HTTP Strict Transport Security) enabled
- No sensitive data in URL parameters

NFR-SEC-003: Payment Security (PCI DSS Compliance)

Priority: High

Description: System shall comply with PCI DSS standards.

Compliance Measures:

- No credit card data stored on platform servers
- Stripe.js for client-side tokenization
- SAQ A (Self-Assessment Questionnaire A) compliance
- Quarterly security scans
- Annual compliance review

Payment Flow:

- Card details never touch platform servers
- Stripe handles all card data
- Payment tokens used for processing
- Secure webhooks for payment notifications

NFR-SEC-004: Protection Against Common Vulnerabilities

Priority: High

Description: System shall protect against OWASP Top 10 vulnerabilities.

Protection Measures:

- SQL Injection: Parameterized queries, ORM usage
- XSS (Cross-Site Scripting): Input sanitization, output encoding
- CSRF (Cross-Site Request Forgery): CSRF tokens
- Authentication: Secure session management
- Sensitive Data Exposure: Encryption, HTTPS
- Access Control: Role-based permissions
- Security Misconfiguration: Regular security audits
- Insecure Deserialization: Input validation
- Logging: No sensitive data in logs
- Insufficient Logging: Comprehensive audit trails

NFR-SEC-005: API Security

Priority: High

Description: System shall secure all API endpoints.

API Security:

- API authentication: JWT bearer tokens
- Rate limiting: 100 requests per minute per user
- CORS policy: Restrict to frontend domain
- Input validation: Validate all API inputs
- Error messages: No sensitive information leaked

NFR-SEC-006: Audit and Monitoring

Priority: High

Description: System shall log and monitor security events.

Logging:

- All authentication attempts (success/failure)
- All financial transactions
- Admin actions (campaign approval, transfers)
- Data access and modifications
- Security events (suspicious activity)

Monitoring:

- Real-time alerts for suspicious activity
- Failed login attempt tracking (lockout after 5 attempts)
- Unusual transaction patterns
- Sentry for error tracking

NFR-SEC-007: Privacy and Data Protection

Priority: High

Description: System shall comply with data protection regulations.

Compliance:

- GDPR compliance for EU users
- CCPA compliance for California users
- Privacy policy clearly stated
- Cookie consent banner
- User data export capability
- Right to be forgotten (account deletion)

Data Handling:

- Minimal data collection (only necessary fields)
- User consent for data processing
- Anonymization of donor data (if requested)
- Secure data deletion

5.4 User Documentation

NFR-UD-001: User Guides

Priority: Medium

Description: System shall provide comprehensive user documentation.

Student Guide:

- How to register and verify account
- How to create a campaign
- Using the AI campaign assistant
- Best practices for fundraising
- How to track donations
- What happens after campaign completes

Donor Guide:

- How to register
- How to browse and search campaigns
- How to make a donation
- Understanding donation receipts
- How to set up recurring donations
- Managing your donor account

Admin Guide:

- Admin dashboard overview
- Reviewing and approving campaigns
- Verifying student documents
- Processing bank transfers
- Handling disputes and issues
- Generating reports

Formats:

- Online help center (web-based)
- PDF downloadable guides
- Video tutorials (future)
- FAQs

NFR-UD-002: In-App Help

Priority: Low

Description: System shall provide contextual help within the application.

Help Features:

- Tooltips for form fields
- Help icons (?) with pop-up explanations
- Getting started wizard for new users
- Onboarding tutorial for students
- Error messages with helpful suggestions

NFR-UD-003: API Documentation

Priority: Low

Description: System shall provide API documentation for developers.

Documentation:

- API endpoint reference
- Request/response examples
- Authentication instructions
- Error codes and meanings
- Rate limiting information

Format:

- Swagger/OpenAPI specification
- Interactive API explorer

6. Assumptions and Dependencies

6.1 Assumptions

A-001: User Assumptions

- Users have access to stable internet connection
- Users have basic computer literacy
- Users have email addresses
- Students are enrolled in recognized educational institutions
- Donors have valid payment methods (credit/debit cards)
- Users have modern web browsers (last 2 major versions)

A-002: Educational Institution Assumptions

- Universities have bank accounts capable of receiving ACH or Wire transfers
- Universities will cooperate with fund crediting process
- Universities will respond to fund transfer notifications
- University enrollment verification documents are available

A-003: Technical Assumptions

- Stripe API will remain stable and available
- OpenAI/Anthropic API will remain accessible and affordable
- AWS services will maintain 99.9% uptime
- Third-party APIs (email, payment) will not significantly change
- Internet banking infrastructure will remain stable

A-004: Business Assumptions

- There is market demand for education crowdfunding
- Students will use the platform if available
- Donors are willing to contribute to student education
- 10% platform fee is acceptable to users
- Donors trust institutional disbursement model

A-005: Regulatory Assumptions

- Current financial regulations allow this business model
- PCI DSS compliance is achievable with Stripe integration
- GDPR/CCPA compliance is achievable with standard measures
- No significant regulatory changes during development

6.2 Dependencies

D-001: External Service Dependencies

Stripe (Critical Dependency):

- **Dependency:** Payment processing and bank transfers
- **Impact if unavailable:** No donations can be processed, no transfers can be made
- **Mitigation:** Monitor Stripe status, have manual backup process for critical transfers
- **Version:** Latest stable API version

OpenAI or Anthropic (High Dependency):

- **Dependency:** AI campaign content generation
- **Impact if unavailable:** Campaign assistant feature unavailable, but platform functional
- **Mitigation:** Fallback to manual campaign creation, use cached responses for common requests
- **Version:** GPT-4 or Claude API latest version

AWS (Critical Dependency):

- **Dependency:** Hosting, database, file storage
- **Impact if unavailable:** Complete system downtime
- **Mitigation:** Multi-availability zone deployment, automated backups, disaster recovery plan
- **Services:** EC2, RDS, S3, CloudFront, SES

SendGrid/Resend/AWS SES (High Dependency):

- **Dependency:** Email notifications
- **Impact if unavailable:** No email notifications sent
- **Mitigation:** Queue emails for retry, have backup email service
- **Version:** Latest stable API

Sentry (Medium Dependency):

- **Dependency:** Error tracking and monitoring
- **Impact if unavailable:** No automated error tracking, manual monitoring required
- **Mitigation:** Acceptable short-term loss, implement fallback logging

D-002: Team Dependencies

Development Team:

- Availability of 3 team members throughout 19-week project
- Technical skills in React, Node.js, PostgreSQL, Stripe integration

- Ability to learn and implement AI integration

Project Manager:

- Availability of Dr. Ilyas for oversight and approvals
- Timely decision-making on scope and priorities

Project Advisor:

- Availability of Sir Faisal Shahzad for technical guidance
- Regular review sessions

D-003: Infrastructure Dependencies**Internet Connectivity:**

- Stable internet for development team
- Reliable hosting provider uptime
- CDN for global content delivery

Development Tools:

- Access to development software licenses
- GitHub for version control
- Cloud development environments

D-004: Third-Party Integration Dependencies**University Banking Information:**

- Ability to obtain accurate banking details from universities
- University cooperation in providing information
- Validation of banking information accuracy

Document Verification:

- Quality of student ID and enrollment letter uploads
- Availability of university verification contacts (if needed)

D-005: Regulatory Dependencies**Payment Processing Regulations:**

- Compliance with financial regulations
- Adherence to PCI DSS standards (via Stripe)
- No significant regulatory changes during project

Data Protection Regulations:

- GDPR compliance for EU users (if applicable)
- CCPA compliance for California users
- Privacy policy legal review

D-006: Timeline Dependencies**Project Phases:**

- Each phase depends on completion of previous phase
- Testing depends on completion of all features
- Deployment depends on successful testing

Critical Path Items:

- Stripe integration must be completed before payment testing
- Database schema must be finalized before full development
- AI integration must be completed before AI feature testing

D-007: Budget Dependencies**API Costs:**

- Budget for AI API usage (OpenAI/Anthropic)
- Budget for cloud hosting (AWS)
- Budget for email service (SendGrid/SES)
- Budget for monitoring tools (Sentry)

Payment Fees:

- Stripe processing fees: 2.9% + \$0.30 per transaction
- Bank transfer fees: \$0.20-\$1.50 (ACH), \$15-\$30 (Wire)

7. References

Academic and Research Papers

Ref-001: Mollick, E. (2014). "The dynamics of crowdfunding: An exploratory study." *Journal of Business Venturing*, 29(1), 1-16.

Source: Academic journal database

Ref-002: Gerber, E. M., & Hui, J. (2013). "Crowdfunding: Motivations and deterrents for participation." *ACM Transactions on Computer-Human Interaction*, 20(6), 1-32.

Source: ACM Digital Library

Ref-003: Belleflamme, P., Lambert, T., & Schwienbacher, A. (2014). "Crowdfunding: Tapping the right crowd." *Journal of Business Venturing*, 29(5), 585-609.

Source: Academic journal database

Industry Reports

Ref-004: World Bank Group. (2023). "Crowdfunding and Financial Inclusion." *World Bank Publications*.

Source: <https://www.worldbank.org>

Ref-005: Pew Research Center. (2021). "The State of Online Charitable Giving."

Source: <https://www.pewresearch.org>

Ref-006: Statista Research Department. (2024). "Global Crowdfunding Market Size." *Statista Digital Market Outlook*.

Source: <https://www.statista.com>

Technical Documentation

Ref-007: Stripe API Documentation. (2024). "Payment Integration Guide."

Source: <https://stripe.com/docs>

Date: Accessed October 2025

Ref-008: OpenAI Documentation. (2024). "GPT-4 API Reference."

Source: <https://platform.openai.com/docs>

Date: Accessed October 2025

Ref-009: Anthropic Documentation. (2024). "Claude API Guide."

Source: <https://docs.anthropic.com>

Date: Accessed October 2025

Ref-010: React.js Documentation. (2024). "Building User Interfaces."

Source: <https://react.dev>

Date: Accessed October 2025

Ref-011: Next.js Documentation. (2024). "The React Framework for the Web."

Source: <https://nextjs.org/docs>

Date: Accessed October 2025

Ref-012: Node.js Documentation. (2024). "Node.js API Reference."

Source: <https://nodejs.org/docs>

Date: Accessed October 2025

Ref-013: PostgreSQL Documentation. (2024). "PostgreSQL 14 Documentation."

Source: <https://www.postgresql.org/docs>

Date: Accessed October 2025

Ref-014: AWS Documentation. (2024). "Cloud Architecture Best Practices."

Source: <https://aws.amazon.com/architecture>

Date: Accessed October 2025

Ref-015: Redis Documentation. (2024). "Redis Documentation."

Source: <https://redis.io/docs>

Date: Accessed October 2025

AI and Machine Learning Resources

Ref-016: Vaswani, A., et al. (2017). "Attention Is All You Need." *Advances in Neural Information Processing Systems*.

Source: NeurIPS proceedings

Ref-017: Devlin, J., et al. (2019). "BERT: Pre-training of Deep Bidirectional Transformers." *NAACL*.

Source: NAACL proceedings

Ref-018: Zhou, Z. H. (2021). *Machine Learning*. Springer.

Source: Springer publications

Similar Platforms Analyzed

Ref-019: GoFundMe. (2024). "Platform Features and Guidelines."

Source: <https://www.gofundme.com>

Date: Accessed October 2025

Ref-020: Kiva. (2024). "Microlending Platform Overview."

Source: <https://www.kiva.org>

Date: Accessed October 2025

Ref-021: DonorsChoose. (2024). "Education Crowdfunding Model."

Source: <https://www.donorschoose.org>

Date: Accessed October 2025

Books

Ref-022: Dresner, S. (2014). *Crowdfunding: A Guide to Raising Capital on the Internet*. Hoboken, NJ: Wiley.

Source: Wiley publications

Ref-023: Sommerville, I. (2016). *Software Engineering* (10th ed.). Boston: Pearson.
Source: Pearson Education

Ref-024: Géron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* (3rd ed.). O'Reilly Media.
Source: O'Reilly Media

Security and Compliance Standards

Ref-025: PCI Security Standards Council. (2024). "Payment Card Industry Data Security Standard (PCI DSS) v4.0."
Source: <https://www.pcisecuritystandards.org>

Ref-026: OWASP. (2024). "OWASP Top Ten Web Application Security Risks."
Source: <https://owasp.org/www-project-top-ten>

Ref-027: GDPR. (2018). "General Data Protection Regulation."
Source: <https://gdpr.eu>

Expert Consultations

Ref-028: Dr. Sarah Ahmed, Professor of Computer Science, University of Technology
Email: s.ahmed@university.edu
Consultation: System architecture and scalability
Date: September 2025

Ref-029: Mr. James Wilson, Fintech Consultant, PaymentTech Solutions
Email: j.wilson@paymenttech.com
Consultation: Payment gateway integration and compliance
Date: September 2025

Ref-030: Ms. Maria Garcia, Director of Financial Aid, State University
Email: m.garcia@stateuniv.edu
Consultation: Student verification processes and institutional needs
Date: September 2025

Ref-031: Dr. Michael Chen, AI Research Scientist, Tech Innovation Lab
Email: m.chen@techinnovation.com
Consultation: AI/ML integration strategies
Date: September 2025

Project Documents

Ref-032: Project Proposal
Document ID: SEFP-2025-001-PROPOSAL

Date: October 6, 2025

Location: Project repository/Documentation folder

8. Appendices

Appendix A: Glossary

ACH (Automated Clearing House): Electronic network for financial transactions in the United States, typically used for bank-to-bank transfers. Takes 2-3 business days.

API (Application Programming Interface): Set of protocols and tools for building software applications, allowing different systems to communicate.

Campaign: A fundraising initiative created by a student to raise money for educational expenses.

Donor: An individual or organization that contributes money to support a student's campaign.

Escrow: A financial arrangement where funds are held by a third party (the platform) until certain conditions are met (campaign completion).

Institutional Disbursement: The process of transferring raised funds directly to the student's educational institution rather than to the student.

JWT (JSON Web Token): A compact, URL-safe means of representing claims to be transferred between two parties, used for authentication.

OAuth 2.0: An authorization framework that enables applications to obtain limited access to user accounts on an HTTP service.

PCI DSS: Payment Card Industry Data Security Standard - security standards for organizations that handle credit card information.

Stripe: A payment processing platform that handles credit card transactions and bank transfers.

Wire Transfer: Electronic transfer of funds between banks or financial institutions, typically completed in 1 business day but with higher fees than ACH.

Appendix B: Use Case Diagrams

[Note: Use case diagrams would be included here in the actual document. For this text version, they are described.]

Primary Use Cases:

UC-001: Student Campaign Creation

- **Actor:** Student
- **Precondition:** Student is registered and verified
- **Flow:** Create campaign → Use AI assistant → Submit for review → Admin approves → Campaign goes live

UC-002: Donor Makes Donation

- **Actor:** Donor
- **Precondition:** Donor is registered
- **Flow:** Browse campaigns → Select campaign → Enter amount → Process payment → Receive receipt

UC-003: Admin Reviews Campaign

- **Actor:** Admin
- **Precondition:** Campaign submitted for review
- **Flow:** View campaign → Check documents → Approve/Reject → Notify student

UC-004: Admin Processes Transfer

- **Actor:** Admin
- **Precondition:** Campaign completed
- **Flow:** Review completion → Calculate transfer amount → Initiate bank transfer → Track status → Confirm completion

Appendix C: Entity Relationship Diagram (ERD)

[Note: Detailed ERD would be included here. Key entities are listed below.]

Primary Entities:

- Users
- Students
- Donors
- Admins
- Universities
- Campaigns
- Donations
- BankTransfers
- StudentAccountCredits
- Notifications
- Comments

Key Relationships:

- User → Student (1:1)
- User → Donor (1:1)
- User → Admin (1:1)
- Student → University (many:1)
- Student → Campaign (1:many)
- Campaign → Donation (1:many)
- Donor → Donation (1:many)
- Campaign → BankTransfer (1:1)
- University → BankTransfer (1:many)

Appendix D: User Interface Mockups

[Note: UI mockups would be included here. Key screens listed.]

Key Screens:

1. Homepage with featured campaigns
2. Student registration and verification
3. Campaign creation wizard
4. AI campaign assistant interface
5. Campaign detail page
6. Donation flow
7. Student dashboard
8. Donor dashboard
9. Admin dashboard
10. Campaign review interface
11. Transfer processing interface

Appendix E: Database Schema

[Note: Detailed schema would be provided in actual document. Sample table structure below.]

Sample Table: campaigns

```
CREATE TABLE campaigns (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  student_id UUID NOT NULL REFERENCES students(id),
  university_id UUID NOT NULL REFERENCES universities(id),
  title VARCHAR(255) NOT NULL,
  description TEXT NOT NULL,
  goal_amount DECIMAL(10, 2) NOT NULL,
  raised_amount DECIMAL(10, 2) DEFAULT 0,
  status VARCHAR(50) DEFAULT 'draft',
  category VARCHAR(100) NOT NULL,
  start_date DATE,
```

```
end_date DATE,  
created_at TIMESTAMP DEFAULT NOW(),  
updated_at TIMESTAMP DEFAULT NOW()  
);
```

Appendix F: API Endpoint Reference

[Note: Complete API documentation would be provided. Sample endpoints below.]

Authentication Endpoints:

- `POST /api/auth/register` - Register new user
- `POST /api/auth/login` - User login
- `POST /api/auth/logout` - User logout
- `POST /api/auth/forgot-password` - Request password reset

Campaign Endpoints:

- `GET /api/campaigns` - List all campaigns
- `GET /api/campaigns/:id` - Get campaign details
- `POST /api/campaigns` - Create new campaign
- `PUT /api/campaigns/:id` - Update campaign
- `DELETE /api/campaigns/:id` - Delete campaign

Donation Endpoints:

- `POST /api/donations` - Process donation
- `GET /api/donations/:id` - Get donation details
- `GET /api/users/:id/donations` - User's donations

Admin Endpoints:

- `GET /api/admin/campaigns/pending` - Pending campaigns
- `POST /api/admin/campaigns/:id/approve` - Approve campaign
- `POST /api/admin/campaigns/:id/reject` - Reject campaign
- `GET /api/admin/transfers/ready` - Ready for transfer
- `POST /api/admin/transfers` - Initiate transfer

Appendix G: Error Codes

Common Error Codes:

Code	Message	Description
400	Bad Request	Invalid input parameters
401	Unauthorized	Authentication required
403	Forbidden	Insufficient permissions
404	Not Found	Resource not found
409	Conflict	Duplicate resource
422	Unprocessable Entity	Validation failed
429	Too Many Requests	Rate limit exceeded
500	Internal Server Error	Unexpected server error
503	Service Unavailable	Service temporarily down

Appendix H: Testing Strategy

Testing Levels:

1. Unit Testing

- Framework: Jest
- Coverage target: 80%
- Test all functions and components

2. Integration Testing

- Test API endpoints
- Test database operations
- Test third-party integrations

3. System Testing

- End-to-end user workflows
- Payment processing
- Bank transfer simulation

4. Security Testing

- Penetration testing
- Vulnerability scanning
- Authentication testing

5. Performance Testing

- Load testing (JMeter)
- Stress testing
- Response time validation

6. User Acceptance Testing

- Beta testing with real users
- Feedback collection
- Issue resolution

Appendix I: Deployment Checklist

Pre-Deployment:

- ☐ All tests passing
- ☐ Code reviewed and approved
- ☐ Database migrations prepared
- ☐ Environment variables configured
- ☐ SSL certificates installed
- ☐ Domain configured
- ☐ CDN configured
- ☐ Backup systems verified

Deployment:

- ☐ Database backup created
- ☐ Deploy to staging environment
- ☐ Test staging environment
- ☐ Deploy to production
- ☐ Verify production deployment
- ☐ Monitor for errors

Post-Deployment:

- ☐ Smoke testing on production
- ☐ Monitor performance metrics
- ☐ Check error rates
- ☐ Verify third-party integrations
- ☐ Document any issues
- ☐ Communicate with stakeholders

Appendix J: Support and Maintenance Plan

Support Tiers:

1. **Critical Issues** (Response: 1 hour)
 - Payment processing down
 - Site completely inaccessible
 - Data loss or corruption
2. **High Priority** (Response: 4 hours)
 - Major feature broken
 - Security vulnerability
 - Performance degradation
3. **Medium Priority** (Response: 24 hours)
 - Minor feature not working
 - UI/UX issues
 - Non-critical bugs
4. **Low Priority** (Response: 1 week)
 - Enhancement requests
 - Documentation updates
 - Minor improvements

Maintenance Schedule:

- Daily: Automated backups, monitoring checks
- Weekly: Security updates, performance review
- Monthly: Database optimization, dependency updates
- Quarterly: Security audit, disaster recovery test
- Annually: Major version updates, infrastructure review

END OF DOCUMENT

Document Version: 1.0

Last Updated: October 6, 2025

Next Review Date: November 6, 2025

Approved by:

Dr. Ilyas (Project Manager)

Sir Faisal Shahzad (Project Advisor)

Umme Habiba (Team Lead)